

Laporan Praktikum Kontrol Cerdas Week 4

Nama : Zibran Rizki Arta
NIM : 224308049
Kelas : TKA-6B
Akun Github : <https://github.com/Zibranrizki25>
Student Lab Assistant : Mba Salma Halizah

1. Judul Percobaan: *Reinforcement Learning for Autonomous Control*

2. Tujuan Percobaan:

Tujuan dari percobaan *Reinforcement Learning for Autonomous Control* adalah:

- Memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
- Menggunakan *OpenAI Gym* sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori:

Kecerdasan buatan atau *Artificial Intelligence* (AI) adalah sistem yang mempelajari bagaimana membuat komputer dapat berpikir, belajar, dan bertindak seperti manusia (Dosari & Abouellail, 2023). *Reinforcement Learning* (RL) adalah metode pembelajaran di mana agen belajar mengambil keputusan melalui interaksi dengan lingkungan. Agen memperoleh *reward* atau *penalty* sebagai umpan balik dari tindakannya, sehingga ia dapat mengoptimalkan strategi untuk mencapai tujuan tertentu.

Pada dasarnya memungkinkan mesin atau agen mempelajari tindakannya berdasarkan timbal balik dari lingkungan sehingga dapat beradaptasi dengan lingkungan seiring dengan berjalannya waktu. Prinsip yang digunakan dalam

metode ini didasarkan pada teori *reinforcement* yang pada intinya adalah ‘konsekuensi mempengaruhi tindakan’. Ada 3 prinsip dasar pada teori reinforcement yakni konsekuensi yang berakibat baik mendorong terjadinya tindakan, konsekuensi yang berakibat buruk mendorong berkurangnya tindakan dan konsekuensi yang tidak ada dampaknya tidak mempengaruhi tindakan (t.t.).

Deep Q-Network (DQN) adalah algoritma pembelajaran penguatan yang revolusioner, menggabungkan kekuatan *Q-Learning* dan *Deep Learning* untuk mengatasi keterbatasan *Q-Learning* tradisional. *Q-Learning*, yang berfokus pada pembelajaran kebijakan optimal melalui *Q-table*, menjadi tidak efisien dalam lingkungan dengan state yang kompleks atau kontinu. DQN mengatasi masalah ini dengan menggunakan jaringan saraf tiruan untuk memperkirakan nilai Q, memungkinkan penanganan input berdimensi tinggi seperti gambar dan state yang sangat banyak. Dengan teknik seperti *experience replay* dan *target network*, DQN meningkatkan stabilitas dan efisiensi pembelajaran, menjadikannya alat yang ampuh untuk berbagai aplikasi seperti permainan video, robotika, dan mobil otonom (Purba dkk., 2022).

Penerapan *Reinforcement Learning* (RL) telah berkembang menjadi untuk sistem kontrol otonom yang cerdas dan adaptif. Penerapannya mencakup berbagai bidang, mulai dari kendaraan otonom yang dilatih untuk mengambil keputusan navigasi kompleks dalam kondisi lalu lintas dinamis, hingga robotika yang memungkinkan robot melakukan tugas-tugas sulit seperti manipulasi objek dan navigasi di lingkungan yang tidak terstruktur. *Reinforcement Learning* (RL) juga digunakan untuk mengoptimalkan sistem kontrol industri, memungkinkan adaptasi terhadap perubahan kondisi dan peningkatan efisiensi, serta dalam kontrol UAV untuk tugas-tugas seperti pengawasan dan pemetaan, di mana UAV belajar terbang secara otonom di lingkungan yang kompleks.

Software yang digunakan dalam percobaan ini berupa Python dan Open CV. OpenCV (*Open Source Computer Vision Library*) adalah perpustakaan perangkat lunak yang dirancang khusus untuk pengolahan citra dan visi komputer. Dikembangkan oleh Intel dan sekarang bersifat *open-source*,

OpenCV menyediakan berbagai alat dan fungsi yang memudahkan pengembang untuk membuat aplikasi berbasis pengolahan citra dan visi komputer. Python, sebagai salah satu bahasa pemrograman yang didukung oleh OpenCV, menjadi pilihan populer karena sintaksnya yang mudah dipahami dan kemampuannya untuk integrasi dengan berbagai pustaka lain, seperti NumPy, SciPy, dan scikit-learn. Dengan OpenCV Python, pengembang dapat mengakses berbagai algoritma canggih untuk tugas-tugas seperti deteksi objek, pelacakan, pengenalan wajah, dan segmentasi citra. Sehingga bahasa pemrograman Python yang diprogram memiliki keahlian yaitu memberikan hasil keluaran sesuai input dari dataset, metode dan algoritma yang digunakan (Zebua & Rosyani, 2024).

4. Analisis dan Diskusi:

- Analisis

Pada praktikum minggu keempat ini percobaan yang dilakukan adalah mengimplementasikan agen *Reinforcement Learning* (RL) menggunakan algoritma Deep Q-Network (DQN). Tahap awal yang dilakukan adalah membuat file **dn_agent.py** yang kemudian diisikan kode-kode program untuk implementasi *Reinforcement Learning* (RL) menggunakan algoritma Deep Q-Network (DQN). Pertama definisikan sebuah kelas bernama DQNAgentclass dengan kode **classDQNAgent:** yang berfungsi membuat kode program untuk mengimplementasikan agen pembelajaran penguatan *Reinforcement Learning* (RL) yang menggunakan algoritma Deep Q-Network (DQN). Selanjutnya, didalam kelas tersebut terbagi menjadi beberapa bagian antara lain **def__init__(self, state_size, action_size):** adalah sebuah konstruktor kelas, yang dipanggil saat objek DQNAgent dibuat. Fungsinya yaitu untuk menginisialisasi atribut-atribut penting agen, seperti ukuran keadaan (*state_size*) dan ukuran tindakan (*action_size*), serta membangun model jaringan saraf tiruan yang akan digunakan oleh agen. Lalu, **def_build_model(self):** adalah metode yang bertanggung jawab untuk mendefinisikan arsitektur jaringan saraf tiruan yang akan digunakan untuk memperkirakan nilai Q. Kemudian, **def remember(self, state, action, reward, next_state, done):** adalah metode

yang digunakan untuk menyimpan pengalaman agen ke dalam memori *replay*. Pengalaman ini terdiri dari keadaan saat ini, tindakan yang diambil, imbalan yang diterima, keadaan berikutnya, dan apakah episode selesai. Kemudian, **def act(self, state):** adalah metode yang menentukan tindakan apa yang akan diambil oleh agen berdasarkan keadaan saat ini. Metode ini biasanya menggunakan strategi *epsilon-greedy* untuk menyeimbangkan eksplorasi dan eksploitasi. Terakhir, **def replay(self, batch_size):** adalah metode yang melatih jaringan saraf tiruan dengan mengambil sampel acak dari memori *replay*. Metode ini menghitung kerugian (*loss*) dan memperbarui bobot model menggunakan algoritma optimisasi seperti Adam. Secara keseluruhan, metode-metode ini bekerja sama untuk memungkinkan agen DQN belajar dan meningkatkan kinerjanya dalam lingkungan yang diberikan.

Kedua, menentukan apakah sebuah skrip dijalankan sebagai program utama atau diimpor sebagai modul dengan menggunakan kode **if __name__ == '__main__':** yang berfungsi memastikan bahwa kode pelatihan hanya dijalankan ketika skrip dijalankan secara langsung. Kemudian, membuat kode program untuk memulai perulangan yang akan mengulangi blok kode di dalamnya sebanyak jumlah episode yang diinputkan. Dengan menggunakan kode **for e in range(episodes):** berfungsi untuk melatih agen selama beberapa episode, yang memungkinkan agen untuk belajar dan meningkatkan kinerjanya seiring waktu.

Tahap selanjutnya yang dilakukan adalah membuat file **test_agent.py** untuk menguji agen yang telah dilatih pada *environment*. Kemudian, lingkungan **CartPole-v1** dibuat, dan ukuran *state* dan *action* diidentifikasi untuk menginisialisasi agen DQN. Agen DQN diinisialisasi dengan epsilon yang sangat kecil untuk meminimalkan eksplorasi dan memaksimalkan eksploitasi, yang berarti agen akan cenderung memilih tindakan berdasarkan apa yang telah dipelajarinya. Program kemudian menjalankan episode pengujian sesuai jumlah yang diinputkan. Di setiap episode, lingkungan direset, *state* awal direshape, dan agen berinteraksi

dengan lingkungan selama maksimal langkah yang diinputkan. Di setiap langkah, lingkungan dirender, agen memilih tindakan berdasarkan *state* saat ini, dan lingkungan memberikan *state* berikutnya, *reward*, dan *status* selesai. *State* berikutnya direshape untuk *input* agen. Jika *episode* selesai (CartPole jatuh), skor episode dicetak, dan episode berakhir. Setelah semua episode pengujian selesai, lingkungan ditutup.

Kemudian, dilakukan modifikasi program dengan menambahkan teknik **target network** untuk meningkatkan stabilitas pelatihan program serta menguji agen pada *environment* lain seperti **mountaincar-v0** untuk dibandingkan performanya. Untuk modifikasi yang pertama adalah dengan menambahkan visualisasi grafis dari lingkungan **CartPole-v1** dengan kode `gym.make('CartPole-v1', render_mode='rgb_array')` menggunakan **Pygame** dengan kode `def init_pygame(): | def render_pygame` dan menampilkan grafik *score* dan nilai epsilon secara *real time* menggunakan matplotlib dengan kode `plt.ion() | plt.grid() | plt.ioff() | plt.show()`. Untuk modifikasi yang kedua adalah meningkatkan stabilitas pelatihan program pada *environment* **MountainCar-v0**. Menggunakan kode program yang kurang lebih sama dengan kode program modifikasi **CartPole-v1** yang membedakan adalah lingkungan permainan pada modifikasi ini diubah dengan **MountainCar-v0** yang bertujuan untuk menggerakkan mobil ke puncak bukit menggunakan kode `gym.make('MountainCar-v0', render_mode='rgb_array')`, menambahkan kode **steps_list** dan menampilkan jumlah langkah dalam grafik, menambahkan kode **update_plot()**: memiliki fungsi untuk mengupdate plot secara dinamis. Pada modifikasi menggunakan **MountainCar-v0** juga ditambahkan visualisasi grafik yang lebih lengkap dengan menampilkan jumlah langkah per episode.

Perbedaan hasil yang ditampilkan oleh kedua kode antara lain pada program **CartPole-v1** karakteristiknya memiliki ruang keadaan dan tindakan yang relatif sederhana, dimana hasil yang diperoleh adalah agen belajar dengan cepat untuk menjaga tiang tetap tegak dalam waktu yang lama. Sedangkan program **MountainCar-v0** karakteristiknya memiliki

ruang keadaan dan tindakan yang lebih kompleks, dengan tantangan mencapai tujuan, dimana hasil yang diperoleh adalah agen memerlukan lebih banyak episode dan eksplorasi untuk belajar mencapai tujuan. Perbedaan waktu yang dibutuhkan untuk menjalankan kedua program tersebut dapat dipengaruhi oleh berbagai faktor, baik dari sisi perangkat keras (*hardware*) maupun perangkat lunak (*software*). Dari sisi perangkat keras (*hardware*) antara lain prosesor (CPU), memori (RAM), dan kartu grafis (GPU). Sedangkan dari sisi perangkat lunak (*software*) antara lain sistem operasi, optimasi kode, dan bahasa pemrograman yang digunakan.

- Diskusi

Metode implementasi agen *Reinforcement Learning* (RL) memiliki perbedaan mendasar dengan *supervised learning* dalam konteks sistem kendali terletak pada jenis data, tujuan, metode pembelajaran, interaksi, dan aplikasi. *Supervised learning* mengandalkan data berlabel untuk memprediksi output, sementara *Reinforcement Learning* (RL) berinteraksi dengan lingkungan dinamis dan belajar dari *reward*. Tujuan *supervised learning* adalah prediksi, sedangkan *Reinforcement Learning* (RL) adalah optimasi tindakan untuk *reward* maksimum. Strategi eksplorasi dan eksploitasi dalam *Reinforcement Learning* (RL) dapat dioptimalkan dengan teknik seperti *Epsilon-greedy policy*, *Boltzmann Exploration*, *Upper Confidence Bound* (UCB), dan *Thompson Sampling*. Teknik-teknik ini menyeimbangkan antara mencoba tindakan baru dan memanfaatkan strategi yang telah terbukti, memungkinkan agen *Reinforcement Learning* (RL) belajar lebih efisien dan menemukan kebijakan optimal.

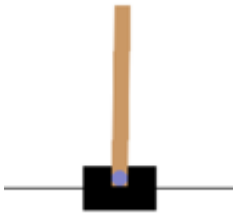
5. Assignment:

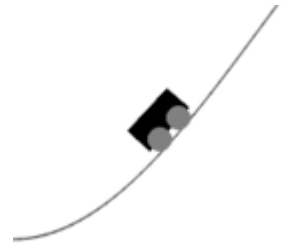
Dalam praktikum yang telah dilakukan mengenai modifikasi program dengan menambahkan teknik **target network** untuk meningkatkan stabilitas pelatihan program serta menguji agen pada *environment* lain seperti **mountaincar-v0** untuk dibandingkan performanya. Agen DQN dirancang untuk belajar mengambil tindakan optimal dalam lingkungan berbasis penguatan (**reinforcement learning**) dengan menggunakan **jaringan saraf tiruan (neural network)** sebagai fungsi perkiraan nilai **Q**. Pada awalnya, agen

mengeksplorasi lingkungan dengan memilih aksi secara acak (epsilon-greedy policy), dan seiring waktu, tingkat eksplorasi (epsilon) berkurang sehingga agen lebih banyak mengeksploitasi pengalaman yang telah dipelajarinya. Jaringan saraf dalam agen ini terdiri dari tiga lapisan: dua lapisan tersembunyi dengan **64 neuron** beraktivasi **ReLU**, serta lapisan keluaran dengan aktivasi **linear** yang menghasilkan nilai **Q** untuk setiap aksi yang memungkinkan. Agen menyimpan pengalaman dalam **replay buffer** menggunakan struktur **deque**, yang memungkinkan pengambilan sampel pengalaman secara acak (experience replay) untuk melatih model dan meningkatkan stabilitas pembelajaran. Selain itu, kode ini menggunakan **target network**, yaitu salinan model utama yang diperbarui setiap beberapa episode untuk mengurangi variansi dalam pembelajaran dan menghindari osilasi nilai **Q**. Proses pembelajaran terjadi dalam **loop episode**, di mana agen mengamati keadaan saat ini (state), memilih aksi, mendapatkan umpan balik (reward), dan memperbarui jaringan saraf berdasarkan pengalaman yang dikumpulkan. Untuk mempercepat pembelajaran, agen diberikan **reward tambahan berdasarkan durasi bertahan dalam episode**, dan **hukuman (penalty) jika gagal mencapai tujuan**. Model diperbarui secara bertahap dengan menggunakan **Mean Squared Error (MSE) loss** dan **Adam optimizer**. Dengan mekanisme ini, agen dapat belajar secara bertahap hingga mampu menyelesaikan tugas dengan lebih efisien.

6. Data dan Output Hasil Pengamatan:

No	Program	Hasil Percobaan	Output
1.	Menggunakan kode program dqn_agent.py tanpa modifikasi	Nilai score yang rendah dan fluktuatif pada hasil running DQN untuk CartPole menunjukkan bahwa agen masih dalam tahap eksplorasi dan belum belajar kebijakan yang stabil. Hal ini bisa disebabkan oleh jumlah episode training yang masih sedikit, nilai epsilon yang masih tinggi, atau model yang belum optimal. Selain itu, replay buffer mungkin belum memiliki cukup pengalaman berkualitas, sehingga pembelajaran agen belum efektif. Untuk meningkatkan performa, bisa dilakukan pelatihan lebih lama (500–1000 episode), penyesuaian epsilon decay agar eksplorasi berkurang lebih lambat, serta modifikasi reward function agar agen lebih terdorong untuk menjaga keseimbangan lebih lama.	<pre>Episode: 1/100, Score: 11, Epsilon: 1.00 Episode: 2/100, Score: 49, Epsilon: 1.00 Episode: 3/100, Score: 18, Epsilon: 0.99 Episode: 4/100, Score: 32, Epsilon: 0.99 Episode: 5/100, Score: 13, Epsilon: 0.99 Episode: 6/100, Score: 13, Epsilon: 0.98 Episode: 7/100, Score: 14, Epsilon: 0.98 Episode: 8/100, Score: 18, Epsilon: 0.97 Episode: 9/100, Score: 23, Epsilon: 0.97 Episode: 10/100, Score: 12, Epsilon: 0.96 Episode: 11/100, Score: 33, Epsilon: 0.96 Episode: 12/100, Score: 16, Epsilon: 0.95 Episode: 13/100, Score: 27, Epsilon: 0.95 Episode: 14/100, Score: 17, Epsilon: 0.94 Episode: 15/100, Score: 13, Epsilon: 0.94</pre>
2.	Menggunakan kode program dqn_agent modify.py	Hasil running DQN untuk MountainCar menunjukkan score yang konsisten di 199, yang menandakan	<pre>Episode: 1/50, Score: 199, Epsilon: 1.00 Episode: 2/50, Score: 199, Epsilon: 0.99 Episode: 3/50, Score: 199, Epsilon: 0.98 Episode: 4/50, Score: 199, Epsilon: 0.97 Episode: 5/50, Score: 199, Epsilon: 0.96 Episode: 6/50, Score: 199, Epsilon: 0.95 Episode: 7/50, Score: 199, Epsilon: 0.94 Episode: 8/50, Score: 199, Epsilon: 0.93 Episode: 9/50, Score: 199, Epsilon: 0.92 Episode: 10/50, Score: 199, Epsilon: 0.91 Episode: 11/50, Score: 199, Epsilon: 0.90 Episode: 12/50, Score: 199, Epsilon: 0.90 Episode: 13/50, Score: 199, Epsilon: 0.89 Episode: 14/50, Score: 199, Epsilon: 0.88 Episode: 15/50, Score: 199, Epsilon: 0.87</pre>

		bahwa agen telah belajar kebijakan optimal untuk mencapai tujuan dalam jumlah langkah maksimum. Dalam MountainCar-v0, score 199 berarti agen tetap bergerak selama 199 langkah, tetapi kemungkinan besar belum berhasil mencapai puncak dalam batas waktu 200 langkah. Hal ini bisa terjadi jika agen hanya berayun tanpa cukup momentum untuk naik ke puncak. Jika tujuan akhir adalah mencapai puncak lebih cepat, dapat dilakukan penyesuaian reward function	
3.	Menggunakan kode program <code>dqn_agent_cartpole.py</code>	Hasil running program test agent untuk CartPole menunjukkan score yang rendah dan konsisten di sekitar 7–9, yang menandakan agen belum belajar	

		mempertahankan keseimbangan dengan baik. Hal ini bisa terjadi karena pelatihan yang kurang optimal, epsilon yang tidak cukup berkurang selama training, atau model tidak menangkap pola permainan dengan baik. Untuk meningkatkan performa, agen perlu dilatih lebih lama, menggunakan replay buffer yang lebih besar, atau menyesuaikan hyperparameter seperti learning rate dan gamma.	
4.	Menggunakan kode program <code>dqn_agent_modifikasi_maountaincar.py</code> (sudah dimodifikasi dengan <code>mountaincar</code>)	Hasil running test agent untuk MountainCar menunjukkan score yang konsisten di 199, yang berarti agen bertahan hingga batas maksimum langkah tanpa mencapai tujuan. Ini menunjukkan bahwa	

		agen mungkin hanya berayun bolak-balik tanpa cukup momentum untuk mencapai puncak. Kemungkinan penyebabnya adalah reward function yang kurang mendorong akselerasi optimal atau model masih perlu perbaikan dalam eksploitasi strategi terbaik.	
--	--	--	--

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat ditarik kesimpulan yaitu:

- Reinforcement Learning (RL) memungkinkan agen untuk belajar mengambil keputusan melalui interaksi dengan lingkungan untuk mencapai tujuan tertentu.
- Algoritma Deep Q-Network (DQN) berhasil diimplementasikan untuk melatih agen dalam lingkungan simulasi OpenAI Gym, dengan penambahan visualisasi Pygame dan plotting Matplotlib untuk pemantauan kinerja.
- Hasil dari modifikasi program menunjukkan bahwa parameter dan lingkungan yang berbeda mempengaruhi performa agen, dengan CartPole-v1 menunjukkan pembelajaran yang lebih cepat dibandingkan MountainCar-v0 yang lebih kompleks.
- Praktikum ini memberikan pemahaman mendalam tentang pentingnya eksplorasi dan eksploitasi dalam RL, serta potensi aplikasinya dalam sistem kendali otonom yang adaptif.

8. Saran:

Untuk meningkatkan performa sistem, sebaiknya menambahkan opsi penyimpanan dan pemuatan model, menerapkan teknik DQN yang lebih canggih, menyediakan penyesuaian parameter melalui argumen baris perintah, menambahkan interaksi pengguna yang lebih baik, serta memberikan opsi untuk mengatur ukuran memori replay. Meningkatkan kompleksitas simulasi dengan lingkungan yang lebih realistis atau penerapan pada perangkat keras fisik seperti robot otonom akan memberikan wawasan lebih lanjut mengenai kesiapan algoritma RL dalam skenario dunia nyata. Dengan menerapkan saran-saran ini, praktikum diharapkan dapat memberikan pemahaman yang lebih mendalam tentang strategi optimasi RL dan aplikasinya dalam sistem kendali otonom.

9. Daftar Pustaka:

Admin, +C007-C014+-+Azhari+

+penerapan+model+reinforcement+learning+bahan+baku+produksi

. (t.t).

Dosari, F. H. M. A., & Abouellail, S. I. A. D. (2023). Artificial Intelligence (AI)

Techniques for Intelligent Control Systems in Mechanical Engineering.

American Journal of Smart Technology and Solutions, 2(2), 55–64.

<https://doi.org/10.54536/ajsts.v2i2.2188>

Purba, R., Lestari, W. S., & Ulina, M. (2022). *Deteksi Serangan DDoS*

Menggunakan Deep Q-Network. 9(1).

Zebua, E. T. P., & Rosyani, P. (2024). *Perancangan Deteksi Objek Kendaraan*

Bermotor Berbasis OpenCV Python menggunakan Metode HOG-SVM

untuk Analisis Lalu Lintas Cerdas. 2(1).